

Inheriting from `std::variant`

Resolving LWG3052

Document #: P2162R0
Date: 2020-05-05
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

§ 1 Introduction

[[LWG3052](#)] describes an under-specification to `std::visit`:

the *Requires* element imposes no explicit requirements on the types in `variants`. Notably, the `variants` are not required to be `variants`. This lack of constraints appears to be simply an oversight.

The original proposal [[P0088R3](#)] makes no mention of other kinds of variants besides `std::variant`, and this does not appear to have been discussed in LEWG.

The proposed resolution in the library issue is to make `std::visit` only work if all of the `variants` are, in fact, `std::variants`:

Remarks: This function shall not participate in overload resolution unless `remove_cvref_t<Variantsi>` is a specialization of `variant` for all $0 \leq i < n$.

This paper suggests a different direction. Instead of restricting to *just* `std::variant` (and certainly not wanting to go all out and design a “variant-like” interface), this paper proposes to allow an additional category of useful types to be `std::visit()`-ed: those that publicly and unambiguously inherit from a specialization of `std::variant`.

§ 2 Inheriting from `variant`

There are two primary motivators for inheriting from `std::variant`.

One is to simply extend functionality. If we’re using `variant` to represent a state machine, we may want additional operations that are relevant to our state that `variant` doesn’t itself provide:

```
struct State : variant<Disconnected, Connecting, Connected>
{
    using variant::variant;

    bool is_connected() const {
        return std::holds_alternative<Connected>(*this);
    }
}
```

```

    friend std::ostream& operator<<(std::ostream&, State const&) {
        // ...
    }
};

```

Another may be to create a recursive variant, as in the example from [\[P1371R2\]](#):

```

struct Expr;

struct Neg {
    std::shared_ptr<Expr> expr;
};

struct Add {
    std::shared_ptr<Expr> lhs, rhs;
};

struct Mul {
    std::shared_ptr<Expr> lhs, rhs;
};

struct Expr : std::variant<int, Neg, Add, Mul> {
    using variant::variant;
};

namespace std {
    template <> struct variant_size<Expr> : variant_size<Expr::variant> {};

    template <std::size_t I> struct variant_alternative<I, Expr> : variant_alternative<I,
        Expr::variant> {};
}

```

That paper even has an example of passing an Expr to `std::visit()` directly, a use-case that this paper is seeking to properly specify. It would be pretty nice if that just worked.

Note also that the example includes an explicit specialization of `variant_size` and `variant_alternative` that just forward along to Expr’s base class. These specializations are pure boilerplate - they basically have to look the way they do, so they don’t really offer much in the way of adding value to the program.

§ 3 Implementation Approach

The proposed resolution of LWG3052 is to, basically, add this constraint onto `std::visit`:

```

template <typename Visitor, typename... Variants>
    requires (is_specialization_of_v<remove_cvref_t<Variants>, variant> && ...)
constexpr decltype(auto) visit(Visitor&&, Variants&&) {
    // as today
}

```

This paper proposes instead that `visit` conditionally upcasts all of its incoming variants to `std::variant` specializations:

```

template <typename Visitor, typename... Variants>
    requires (is_specialization_of_v<remove_cvref_t<Variants>, variant> && ...)
constexpr decltype(auto) visit(Visitor&&, Variants&&) {
    // as today
}

template <typename... Ts>
constexpr auto variant_cast(std::variant<Ts...>& v) -> std::variant<Ts...>& {
    return v;
}
template <typename... Ts>
constexpr auto variant_cast(std::variant<Ts...> const& v) -> std::variant<Ts...> const& {
    return v;
}
template <typename... Ts>
constexpr auto variant_cast(std::variant<Ts...>&& v) -> std::variant<Ts...>&& {
    return std::move(v);
}
template <typename... Ts>
constexpr auto variant_cast(std::variant<Ts...> const&& v) -> std::variant<Ts...> const&&
    {
        return std::move(v);
    }
}

template <typename Visitor, typename... Variants>
constexpr decltype(auto) visit(Visitor&& vis, Variants&&... vars) {
    return visit(std::forward<Visitor>(vis),
        variant_cast(std::forward<Variants>(vars))...);
}

```

This means the body of `std::visit` for implementations can remain unchanged - it, as today, can just assume that all the variants are indeed `std::variants`. Such an implementation would allow visitation of the State and Expr examples provided earlier.

Now, this means we can `std::visit` a variant that we can't even invoke `std::get` or `std::get_if` on, such as with this delightful type courtesy of Tim Song:

```

struct MyEvilVariantBase {
    int index;
    char valueless_by_exception;
};

struct MyEvilVariant : std::variant<int, long>, std::tuple<int>, MyEvilVariantBase { };

```

But... who cares. Don't write types like that.

§ 4 Implementation Experience

The libc++ implementation has supported this design since day one [[libcpp](#)].

The Microsoft STL implementation also already supports this design [[stlstd](#)] since the first Visual Studio 2019 release in April 2019.

§ 5 Wording

Change 20.7.7 [\[variant.visit\]](#):

```
template<class Visitor, class... Variants>
    constexpr see below visit(Visitor&& vis, Variants&&... vars);
template<class R, class Visitor, class... Variants>
    constexpr R visit(Visitor&& vis, Variants&&... vars);
```

- 2 Let *as-variant* denote the exposition-only function template

```
template<class... Ts>
constexpr variant<Ts...>& as-variant(const variant<Ts...>& var) { return var; }
```

Let n be `sizeof...(Variants)`. For each $0 \leq i < n$, let V_i denote the type `remove_cvref_t<decltype(as-variant(vars $i$ ))>`.

- 1 *Constraints*: V_i is a valid type for all $0 \leq i < n$.

- 0 Let VR_i denote the type V_i with the addition of Variant_i 's cv and reference qualifiers. Let VR denote the pack of types VR_i .

- 1 ~~Let n be `sizeof...(Variants)`.~~ Let m be a pack of n values of type `size_t`. Such a pack is called valid if $0 \leq m_i < \text{variant_size_v}<\text{remove_reference_t}<\text{Variants}_i> V_i>$ for all $0 \leq i < n$. For each valid pack m , let $e(m)$ denote the expression:

```
- INVOKE(std::forward<Visitor>(vis), get<m>(std::forward<Variants>(vars))...)
    // see [func.require]
+ INVOKE(std::forward<Visitor>(vis), get<m>(std::forward<VR>(vars))...) // see
    [func.require]
```

for the first form and

```
- INVOKE<R>(std::forward<Visitor>(vis), get<m>(std::forward<Variants>
    (vars))...) // see [func.require]
+ INVOKE<R>(std::forward<Visitor>(vis), get<m>(std::forward<VR>(vars))...) //
    see [func.require]
```

for the second form.

- 2 *Mandates*: For each valid pack m , $e(m)$ is a valid expression. All such expressions are of the same type and value category.
- 3 *Returns*: $e(m)$, where m is the pack for which m_i is ~~`vars $i$ .index()`~~ *as-variant*(vars _{i}).index() for all $0 \leq i < n$. The return type is `decltype(e(m))` for the first form.
- 4 *Throws*: `bad_variant_access` if ~~any variant in vars is valueless by exception()~~ if *as-variant*(vars).valueless_by_exception() || ... is true.
- 5 *Complexity*: For $n \leq 1$, the invocation of the callable object is implemented in constant time, i.e., for $n=1$, it does not depend on the number of alternative types of ~~Variants _{0}~~ V_0 . For $n>1$, the

invocation of the callable object has no complexity requirements.

§ 5.1 Feature-test macro

This paper proposes to bump the value `__cpp_lib_variant`. The macro already exists, so this is, in a sense, free. And users can use the value of this macro to avoid having to specialize `variant_size` and `variant_alternative` for their inherited variants.

§ 6 Acknowledgments

Thanks to Casey Carter, Ville Voutilainen, and the unfortunately non-alliterative Tim Song for design discussion and help with the wording.

§ 7 References

[libcpp] Michael Park. 2017. libcpp variant.

<https://github.com/llvm/llvm-project/blob/24b4965ce65b14ead595dcc68add22ba37533207/libcxx/include/variant#L455>

[LWG3052] Casey Carter. visit is underconstrained.

<https://wg21.link/lwg3052>

[P0088R3] Axel Naumann. 2016. Variant: a type-safe union for C++17 (v8).

<https://wg21.link/p0088r3>

[P1371R2] Sergei Murzin, Michael Park, David Sankel, Dan Sarginson. 2020. Pattern Matching.

<https://wg21.link/p1371r2>

[stlstdl] Microsoft. 2019. stlstdl variant.

<https://github.com/microsoft/STL/blob/65d98ffabab3a95d79255f741daa1230692e8066/stl/inc/variant#L1638-L1657>